FunDM

FunDM runs functional programs, lets you edit their HTML output, and propagates those edits back to the source.

This guide has four parts:

1. **Quick start**: Choose Docker or a local setup to get started.
2. **Interactive editor**: Run the paper's ten benchmarks or write your own programs, directly manipulate their outputs, and propagate the changes back to the source.
3. **Experiments**: Run the performance experiments and inspect the summarized results in the dashboard.
4. **Development**: Build and further develop FunDM itself.

If you only want to use the interactive editor, you can skip the experiment setup and the Experiments section.

I want to...	Docker	Local
Try the interactive editor	Start the container and open the editor	Open the editor directly
Run the experiments	Start the container and open the experiment dashboard	Start the experiment backend

1. Quick start

1.1 Docker

The prebuilt image targets Linux x86-64 (amd64).

Install and start Docker. From the directory containing the prebuilt image archive, load the image and start the container:

```
docker load --input FunDM-image.tar.gz
docker run --rm -p 8000:8000 -p 3000:3000 fundm:artifact
```

Keep the container running and open either page in your browser:

Page	Address	Next step
Interactive editor	http://localhost:8000/ui/index.html	Try the Counter example
Experiments	http://localhost:8000/experiment/index.html	Run the benchmarks

The same container serves both pages. Press **Ctrl+C** in the terminal to stop it.

1.2 Local

1.2.1 Interactive editor

Open ui/index.html in a desktop browser, then follow the Counter example.

1.2.2 Experiments

Install **Elm 0.19.1** (installation guide), **Node.js**, and **npm**. From the artifact root directory, start the backend:

```
cd experiment/backend
npm ci
node server.js
```

Keep this terminal running, then open experiment/index.html in a desktop browser and run the benchmarks.

2. Interactive editor

2.1 Run & direct manipulation

1. Choose **Counter** from the example menu and click **Run**. The counter appears on the right with an initial value of `0`.
2. Click **Increment**. The counter should display `1`.
3. Click **To Edit**. Right-click the number `1` in the output and hover over **Content: int type**.
4. Enter `2` in the number field beside the slider and press **Enter** to apply the edit. The slider can also adjust the value.

2.2 Verify propagation to the source

1. Click **Update Source**. In the source editor, `initCount` changes from `0` to `1`.
2. Click **Run** in the top menu to restart the updated program. The counter now starts at `1`.
3. Click **Increment** again. The counter should display `2`, reproducing the result you edited and confirming that the edit was propagated to the source program.

The example menu contains ten programs. Select another example and click **Run** to try it. Some example illustrations load from the internet.

3. Experiments

3.1 Run the benchmarks

All ten benchmarks are selected by default. Click **Run** and keep the page open until **Experiment completed!** appears. To try only **Counter**, click **Clear All**, select **Counter**, then click **Run**.

Each **Run** clears saved results for all benchmarks, including those not selected.

3.2 Read the summary

The **Experiment Results** table at the top of the page summarizes the results, with one row per selected benchmark. Use it to inspect update times and the extent of source changes. Detailed backward and AST analysis results appear below it.

Each benchmark includes 12 edits: content, structure, and style changes before any event and after one, two, or three events. Each edit is applied independently to the original program five times, and the timings are averaged. All times are in milliseconds.

Column	Meaning
Loc	Number of source lines
Fwd	Mean time to parse, forward-evaluate, and print the original program
Static / 1-Dyn / 2-Dyn / 3-Dyn	Mean backward-update time before any event / after one, two, or three events
Duration Percentage	Group mean backward time divided by the overall mean forward time
Bwd	Mean backward-update time over all edits and repetitions
DHeight	Mean of the per-edit AST difference heights
DLoc	Mean changed-line count, using the larger of inserted and removed line counts

Absolute timings vary with the machine and browser.

3.3 Benchmark reference

Three benchmarks use different names in the interactive editor:

Interactive example	Experiment benchmark
Conference Surplus	Conf Surplus
Meal Ordering	Doodle
House Sailing	Estate Retail

Source programs and predefined deltas are in `experiment/Benchmarks/<benchmark>/`. Saved results, including the updated source, are in its `summaries/<delta>/` subdirectory.

4. Development

Install **Elm 0.19.1** following the official installation guide, then rebuild the editor from the artifact root:

```
elm make src/Ui/Main.elm --output=ui/elm.js
```

Reload the browser page after a build.